

Mikroprocesorová technika

Prednáška č.2: ATmega328P a digitálne I/O

prof. Ing. Ivan Virgala, PhD.

Technická univerzita v Košiciach
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky
ARM-lab (Applied Robotics and Mechatronics Laboratory)





- ① Porty ATmega328P a registre GPIO
- ② Vstup, výstup a interný pull-up
- ③ Bitové operácie a maskovanie
- ④ Aktívna logika a elektrické limity GPIO
- ⑤ LED, tlačidlo a mapovanie Arduino pinov



- Rozlíšiť porty B, C, D a ich piny
- Registre DDRx, PORTx a PINx
- Konfigurácia pinu ako vstup alebo výstup
- Interný pull-up rezistor
- Nastaviť, vynulovať, prepnúť a otestovať bit pomocou masky
- Aktívna logická nulu a aktívna logická jednotku
- Pripojenie LED a tlačidla s ohľadom na elektrické limity
- Namapovať Arduino piny UNO R3 na porty ATmega328P

Vstup

- tlačidlo,
- koncový spínač,
- logický signál zo snímača,
- impulzy z enkodéra.

Výstup

- LED,
- relé cez tranzistor,
- vstup drivera motora,
- riadiaci signál pre perifériu.

Digitálne I/O je základ pre všetky ďalšie témy: prerušenia, časovače, PWM, enkodéry aj komunikáciu.

1. Porty a registre GPIO

Digitálne piny ATmega328P



- Port je skupina digitálnych liniek ovládaných spoločnými registrami.
- ATmega328P má pre bežné GPIO tri porty: PORTB, PORTC, PORTD.
- Jeden bit registra zodpovedá jednej fyzickej linke portu.
- Napríklad bit PB5 reprezentuje piaty bit portu B.

Port	Typické bity	Poznámka
PORTB	PB0–PB7	zahŕňa pin s LED na UNO R3
PORTC	PC0–PC6	PC6 je reset, nie bežné GPIO
PORTD	PD0–PD7	zahŕňa UART piny PD0, PD1



DDR_x

Data Direction Register

- 0: vstup
- 1: výstup

PORT_x

Output / pull-up register

- pri výstupe: logická 0/1
- pri vstupe: pull-up vyp/zap

PIN_x

Input Pins Register

- čítanie skutočnej úrovne na pine
- testovanie vstupu

Smer linky určuje DDR_x. Register PORT_x potom buď nastavuje výstupnú úroveň, alebo zapína interný pull-up rezistor. Stav pinu sa číta z PIN_x.



Úloha

Nastaviť bit PD6 v registri PORTD bez zmeny ostatných bitov.

```
PORTD = 0b00001111;
```

```
PORTD = PORTD | (1 << PD6);
```

```
/* skrátený zápis */
```

```
PORTD |= (1 << PD6);
```

PORTD	=	0b00001111
1 « PD6	=	0b01000000
výsledok	=	0b01001111

Základná myšlienka

 Operácia OR nastaví vybraný bit na 1 a ostatné bity ponechá bez zmeny.



Úloha

Vynulovať bit PD2 v registri PORTD bez zmeny ostatných bitov.

```
PORTD = 0b10001101;
```

```
PORTD = PORTD & ~(1 << PD2);
```

```
/* skrátený zápis */  
PORTD &= ~(1 << PD2);
```

$$\begin{array}{rcl} \text{PORTD} & = & 0b10001101 \\ 1 \ll \text{PD2} & = & 0b00000100 \\ \hline \sim(1 \ll \text{PD2}) & = & 0b11111011 \\ \text{výsledok} & = & 0b10001001 \end{array}$$

Základná myšlienka

 Maska s nulou na požadovanom mieste vynuluje vybraný bit pomocou operácie AND.



Úloha

Prepnúť bit PD5 v registri PORTD: ak bol 1, bude 0; ak bol 0, bude 1.

```
PORTD = 0b10111100;
```

```
PORTD = PORTD ^ (1 << PD5);
```

```
/* skrátený zápis */
```

```
PORTD ^= (1 << PD5);
```

$$\text{PORTD} = 0b10111100$$
$$1 \ll \text{PD5} = 0b00100000$$

$$\text{výsledok} = 0b10011100$$
$$10011100 \oplus 00100000 = 10111100$$

Základná myšlienka

 Operácia XOR prepne vybraný bit a ostatné bity ponechá bez zmeny.

```
1 DDRB |= (1 << PB5); // PB5 ako vystup  
2 DDRB &= ~(1 << PB5); // PB5 ako vstup
```

Pravidlo

- bit v DDR = 0: vstup,
- bit v DDR = 1: výstup.

DDR _{xn}	Režim pinu	Typické použitie
0	vstup	tlačidlo, snímač, externý signál
1	výstup	LED, tranzistor, logické riadenie



```
1 DDRB |= (1 << PB5); // PB5 vystup
2 PORTB |= (1 << PB5); // PB5 = log. 1
3 PORTB &= ~(1 << PB5); // PB5 = log. 0
```

Pri výstupe

- PORTx=0: pin je približne 0 V,
- PORTx=1: pin je približne 5 V,
- GPIO pin neposkytuje presne 0 V a 5 V. Odchýlka od ideálnych hodnôt rastie so zaťažením pinu.



```
1 DDRC  &= ~(1 << PC1); // PC1 ako vstup
2 PORTC &= ~(1 << PC1);  // pull-up vypnutý
3
4 PORTC |=  (1 << PC1);  // pull-up zapnutý
```

Pri vstupe

- PORTx=0: vysoká impedancia,
- PORTx=1: interný pull-up na VCC,
- tlačidlo sa potom zvyčajne pripája proti GND.

Rovnaký register PORTx má iný praktický význam podľa toho, či je pin vstup alebo výstup.



```
1 if (PINC & (1 << PC1))
2 {
3     // na pine PC1 je logicka 1
4 }
5 else
6 {
7     // na pine PC1 je logicka 0
8 }
```

- PINx sa používa na čítanie stavu vstupu.
- Pri výstupe môže PINx ukázať, aká úroveň je fyzicky na pine.



DDRxn	PORTxn	Režim	Praktický stav
0	0	vstup	vysoká impedancia, bez pull-up
0	1	vstup	interný pull-up zapnutý
1	0	výstup	pin tlačí logickú 0
1	1	výstup	pin tlačí logickú 1

Typická chyba

- $PORTx \mid = \text{maska}$ pri vstupe neznamená "zapíš 5 V na pin", ale zapni pull-up.
- $PINx$ nie je register na nastavovanie smeru. Na smer slúži $DDRx$.



Priamy zápis

```
1 PORTB = 0x20;
```

Prepíše celý port B. Môže nechcane zmeniť aj ostatné piny.

Maskovaný zápis

```
1 PORTB |= (1 << PB5);
```

Mení iba bit PB5. Ostatné bity zostanú zachované.

V programe je dôležité meniť iba tie bity, ktoré patria danej úlohe.

2. Bitové operácie

Ako bezpečne meniť jednotlivé bity registrov



- Register GPIO je 8-bitová hodnota.
- Každý bit môže mať hodnotu 0 alebo 1.
- Na prácu s jedným bitom potrebujeme masku.
- Najčastejšia maska má tvar $(1 \ll \text{bit})$.

Bit	Maska v C	Binárne
0	$(1 \ll 0)$	0b00000001
2	$(1 \ll 2)$	0b00000100
5	$(1 \ll 5)$	0b00100000
7	$(1 \ll 7)$	0b10000000



```
1          // 0b000000001
1 << 1     // 0b000000010
1 << 2     // 0b000000100
1 << 5     // 0b00100000
```

Výhoda

- nemusíme ručne počítat binárne konštanty,
- kód je čitateľný,
- symbol PB5 pripomína konkrétny pin.

Maska vyberá konkrétny bit registra. Bez maskovania by sme pri zápise mohli nechcane zmeniť aj ostatné piny portu.



```
1 PORTD |= (1 << PD6);    // nastav bit PD6 na 1
```

Príklad:

```
1 PORTD = 0b00001111;  
2 PORTD |= (1 << PD6);  
3 // vysledok: 0b01001111
```

Princíp

- OR s nulou pôvodný bit nemení.
- OR s jednotkou nastaví vybraný bit na 1.



```
1 PORTD &= ~(1 << PD2);    // vynuluj bit PD2
```

Príklad:

```
1 PORTD = 0b10001101;  
2 PORTD &= ~(1 << PD2);  
3 // vysledok: 0b10001001
```

Princíp

- maska: 00000100,
- negovaná maska: 11111011,
- AND s nulou vynuluje iba vybraný bit.



```
1 PORTD ^= (1 << PD5);    // zmen stav bitu PD5
```

Príklad:

```
1 PORTD = 0b10111100;  
2 PORTD ^= (1 << PD5);  
3 // vysledok: 0b10011100  
4  
5 PORTD ^= (1 << PD5);  
6 // vysledok: 0b10111100
```

Princíp

- XOR s nulou bit nemení.
- XOR s jednotkou bit prepne.
- Vhodné pre blikanie LED.

```
1 if (PINC & (1 << PC1))
2 {
3     // bit PC1 je 1
4 }
5 else
6 {
7     // bit PC1 je 0
8 }
```

- Výsledok výrazu nie je nutne 1; je to buď nula, alebo nenulová maskovaná hodnota.
- V podmienke `if` znamená každá nenulová hodnota `TRUE`.



```
1 PORTD |= (1 << PD5) | (1 << PD4) | (1 << PD3);
```

Úloha	Typický zápis
nastaviť PD3, PD4, PD5	<code>PORTD = maska;</code>
vynulovať PD3, PD4, PD5	<code>PORTD &= ~maska;</code>
prepnúť PD3, PD4, PD5	<code>PORTD $\hat{=}$ maska;</code>

```
1 uint8_t maska = (1 << PD5) | (1 << PD4) | (1 << PD3);  
2 PORTD &= ~maska;
```



Operátor	Význam	Použitie pri registroch
«	posun doľava	vytvorenie masky
	bitový OR	nastavenie bitu
&	bitový AND	testovanie alebo nulovanie bitu
~	bitová negácia	negovaná maska
^	bitový XOR	prepnutie bitu

Pozor na rozdiel

& a | sú bitové operátory. && a || sú logické operátory pre pravdivostné výrazy.



```
1 #define BIT_SET(reg, bit)    ((reg) |=  (1 << (bit)))
2 #define BIT_CLEAR(reg, bit) ((reg) &= ~(1 << (bit)))
3 #define BIT_TOGGLE(reg, bit) ((reg) ^=  (1 << (bit)))
4 #define BIT_READ(reg, bit)   ((reg) &  (1 << (bit)))
```

- Makrá zvyšují čitelnost.



```
1 uint16_t cislo = 674;
2 uint8_t hornyByte;
3 uint8_t dolnyByte;
4
5 hornyByte = cislo >> 8;    // 2
6 dolnyByte = cislo & 0xFF;  // 162
```

Dôvod použitia

- Rovnaké operácie použijeme pri komunikácii.
- 16-bitovú hodnotu často treba preniesť po 8-bitových častiach.
- Princíp maskovania je rovnaký ako pri GPIO.



```
1 uint8_t hornyByte = 2;  
2 uint8_t dolnyByte = 162;  
3 uint16_t cislo;  
4  
5 cislo = ((uint16_t)hornyByte << 8) | dolnyByte;  
6 // cislo = 674
```

- Pretypovanie na `uint16_t` je dobrý zvyk pri posune o 8 bitov.
- Používame typy `uint8_t`, `uint16_t`, aby bola veľkosť hodnoty jednoznačná.

V mikroprocesorovej technike nechceme iba “číslo”, ale presnú bitovú reprezentáciu. Preto sú masky, posuny a celočíselné typy základný nástroj.

3. Elektrická interpretácia GPIO

Logická hodnota nie je automaticky bezpečný prúd



Logická hodnota

- 0 alebo 1 v programe,
- bit v registri,
- výsledok podmienky.

Elektrická úroveň

- napätie na pine,
- prúd tečúci cez záťaž,
- reálne limity súčiastky.

Pri UNO R3 je napájanie ATmega328P 5 V. Logická jednotka preto znamená napätie blízke 5 V, nie presne ideálnych 5 V pri každom zaťažení.



Aktívna jednotka

Signál je aktívny, keď má hodnotu 1.

- `LED_ON = 1`,
- výstup dodáva prúd do záťaže,

Aktívna nula

Signál je aktívny, keď má hodnotu 0.

- `BUTTON_PRESSED = 0`,
- LED svieti pri `PORTx = 0`,
- bežné pri pull-upoch a shieldoch.

V programe treba pomenovať fyzikálny význam signálu. Hodnota 0 nemusí znamenať “neaktívne”.



```
1 DDRC  &= ~(1 << DDC1); // PC1 vstup
2 PORTC |= (1 << PC1);  // interny pull-up zapnuty
3
4 if (!(PINC & (1 << PC1)))
5 {
6     // tlacidlo je stlacene
7 }
```

- Bez stlačenia drží pull-up vstup v logickej 1.
- Pri stlačení je pin spojený s GND, teda čítame logickú 0.
- Preto je podmienka negovaná pomocou !.

Ak je pin nastavený ako vstup bez pull-upu a bez externého pripojenia, jeho stav nie je spoľahlivo definovaný.

- môže reagovať na rušenie,
- môže náhodne meniť stav,
- v programe vzniká dojem “pokazeného” tlačidla.

Pravidlo

Každý digitálny vstup musí mať definovaný stav: interný pull-up, externý pull-up, externý pull-down alebo aktívny zdroj signálu.



- Pin mikrokontroléra nie je napájací zdroj pre výkonové zariadenia.
- Absolútne maximum prúdu cez jeden I/O pin ATmega328P je 40 mA; v návrhu treba používať nižšie hodnoty.
- Celkový prúd cez napájacie vetvy mikrokontroléra je tiež obmedzený.
- Pre LED je vhodné voliť približne 5 mA až 10 mA, nie hraničné hodnoty.

Dôležité

Hodnota z datasheetu “absolute maximum” nie je odporúčaný pracovný bod. Je to hranica, za ktorou hrozí poškodenie alebo skrátenie životnosti.



Source

Pin je v logickej 1 a dodáva prúd do záťaže.

`pin -> LED -> R -> GND`

Sink

Pin je v logickej 0 a prúd tečie zo zdroja cez záťaž do pinu.

`VCC -> R -> LED -> pin`

Oba režimy sú pri AVR použiteľné.



Aktívna jednotka

PB5 -> R -> LED -> GND

- LED svieti pri PB5 = 1.
- Pin generuje prúd.

Aktívna nula

VCC -> R -> LED -> PB5

- LED svieti pri PB5 = 0.
- Pin pohlcuje prúd.

Približný výpočet: pri 5 V, 2 V na LED a 8 mA vychádza $R \approx (5 - 2)/0.008 = 375 \Omega$.
Prakticky zvolíme 330 Ω alebo 470 Ω .



```
1 #define F_CPU 16000000UL
2 #include <avr/io.h>
3 #include <util/delay.h>
4
5 int main(void)
6 {
7     DDRB |= (1 << DDB5);    // PB5 ako vystup
8
9     while (1)
10     {
11         PORTB ^= (1 << PB5); // zmena stavu LED
12         _delay_ms(500);
13     }
14 }
```

- Na Arduino UNO R3 je digitálny pin D13 pripojený k PB5.
- Tento príklad preto bliká aj so zabudovanou LED dosky UNO R3.



```
1 #include <avr/io.h>
2
3 int main(void)
4 {
5     DDRB |= (1 << DDB5);    // PB5 vystup: LED
6     DDRC &= ~(1 << DDC1);   // PC1 vstup: tlacidlo
7     PORTC |= (1 << PC1);    // pull-up na PC1
8
9     while (1)
10    {
11        if (!(PINC & (1 << PC1)))
12            PORTB |= (1 << PB5);    // LED zapni
13        else
14            PORTB &= ~(1 << PB5);   // LED vypni
15    }
16 }
```



Arduino pin	ATmega328P pin	Port/bit	Poznámka
D0	PD0	PD0	UART RX
D1	PD1	PD1	UART TX
D2	PD2	PD2	externé prerušenie INT0
D3	PD3	PD3	PWM, INT1
D4	PD4	PD4	digitálne I/O
D5	PD5	PD5	PWM
D6	PD6	PD6	PWM
D7	PD7	PD7	digitálne I/O



Arduino pin	ATmega328P pin	Port/bit	Poznámka
D8	PB0	PB0	digitálne I/O
D9	PB1	PB1	PWM
D10	PB2	PB2	PWM, SPI SS
D11	PB3	PB3	PWM, SPI MOSI
D12	PB4	PB4	SPI MISO
D13	PB5	PB5	SPI SCK, LED L
A0	PC0	PC0	digitálne I/O alebo ADC0
A1	PC1	PC1	digitálne I/O alebo ADC1



Arduino pin	ATmega328P pin	Port/bit	Poznámka
A2	PC2	PC2	digitálne I/O alebo ADC2
A3	PC3	PC3	digitálne I/O alebo ADC3
A4	PC4	PC4	ADC4, SDA pri TWI/I2C
A5	PC5	PC5	ADC5, SCL pri TWI/I2C
RESET	PC6	PC6	reset, nie bežný GPIO pin

Poznámka pre prax

Piny D0 a D1 sú používané na sériovú komunikáciu s USB prevodníkom.



- ATmega328P používa pre GPIO porty B, C a D.
- Smer pinu nastavuje DDRx: 0 znamená vstup, 1 znamená výstup.
- PORTx pri výstupe nastavuje logickú *úroveň*, pri vstupe zapína interný pull-up.
- PINx slúži na čítanie skutočného stavu pinu.
- Jednotlivé bity meníme maskami: |, &, ~, ^, «.
- Aktívna nula je bežná pri tlačidlách s pull-upom a pri niektorých LED zapojeniach.
- GPIO má elektrické limity; LED musí mať rezistor a výkonové záťaže sa nepripájajú priamo.



- ① Aký je rozdiel medzi DDRB, PORTB a PINB?
- ② Čo urobí príkaz `DDRB |= (1 << DDB5);`?
- ③ Čo urobí príkaz `PORTC |= (1 << PC1);`, ak je PC1 vstup?
- ④ Prečo pri čítaní tlačidla s pull-upom často testujeme negovanú podmienku?
- ⑤ Prečo je `PORTB = 0x20;` nebezpečnejšie ako `PORTB |= (1 << PB5);`?



```
1 #define F_CPU 16000000UL
2 #include <avr/io.h>
3 #include <util/delay.h>
4
5 int main(void)
6 {
7     // 1. konfiguracia smeru: DDRx
8     // 2. inicializacia vystupov alebo pull-upov: PORTx
9
10    while (1)
11    {
12        // 3. citanie vstupov: PINx
13        // 4. zmena vystupov: PORTx
14    }
15 }
```

Ďakujem za
pozornosť